

Top 10 web hacking techniques of 2021



James Kettle

Director of Research

[@albinowax](#)



Published: Wednesday, 9 February 2022 at 13:59 UTC

Updated: Thursday, 10 February 2022 at 15:20 UTC



Welcome to the Top 10 (new) Web Hacking Techniques of 2021, the latest iteration of our [annual community-powered effort](#) to identify the most significant web security research released in the last year.

Since kicking off the selection process in January, the infosec community has nominated 40 research papers, then voted on their favourites to whittle the list down to 15 final-round candidates. Finally, an expert panel consisting of myself and noted researchers [Nicolas Grégoire](#), [Soroush Dalili](#), and [Filedescriptor](#) have voted on the 15 finalists to create the official top 10. As usual, we haven't excluded PortSwigger research, but panellists can't vote for anything they're affiliated with.

The research quality this year was frankly exceptional - it's the strongest year I've seen since getting involved back in 2015. This has led to fierce competition for the top 10, and numerous high-standard research papers missing out. I usually name a few of my favourite runners up but this year there's so many it would be unfair - instead I recommend anyone with time explores the [full nomination list](#). Massive thanks to everyone who contributed to this wave of research!

One particular theme dominated this year. In both the nominations and the final top ten, we saw heavy focus on [HTTP Request Smuggling](#), and attacks on parser inconsistency in general. As systems get more complex and more connected, these threats bloom. It'll be interesting to see where the parser battleground shifts when HTTP/1.1 use eventually starts to dwindle in a few decade's time.

Let's begin the countdown!

10 - Fuzzing for XSS via nested parsers

With an age-old topic like [XSS](#), it's all too easy to think you know it all already and glibly dismiss new research. Gems like [Fuzzing for XSS via nested parsers](#) prove just how risky this is. In this fluff-free post, [Psych0tr1a](#) shows how to turn stacked HTML sanitization rules against each other with specular results. Impressive case-studies and a clear, practical methodology cement this as a piece of top-tier research.

9 - HTTP Smuggling via Higher HTTP Versions

At the start of 2021, HTTP/2 was thought to be largely free of security concerns beyond timing attacks and minor DoS concerns. [Emil Lerner's HTTP Smuggling via Higher HTTP Versions](#) destroyed this myth, using custom tooling and innovative techniques to reveal numerous holes in HTTP/2 to HTTP/1.1 conversion. The slide deck is crammed with novel attacks and if you're fluent in Russian, be sure to check out [the presentation](#) too. Emil's also written up some [terrifying new findings on HTTP/3](#) since.

8 - Practical HTTP Header Smuggling

A vulnerability can be prevalent, well-understood and high-impact, but if nobody knows how to detect it, it can be tempting to just... refocus your thoughts on something more profitable. CL.CL request smuggling had been quietly lurking in this niche for quite a while.

In [Practical HTTP Header Smuggling](#), [Daniel Thatcher](#) isolates a core component of HTTP Request Smuggling, and elegantly restiches it into a strategy that makes it possible to identify both CL.CL vulnerabilities, and generic hidden-header attacks, all integrated into [Param Miner](#). In case you have any doubts about just how valuable this methodology is, he illustrates it with multiple case-studies targeting AWS. You'll hear more about this technique in future.

7 - JSON Interoperability Vulnerabilities

JSON has long been known for being a bit quirky, but largely dodged the barrage of exploits affecting XML parsing. However, whatever the format, if you're going to parse something twice, things are going to go wrong.

[JSON Interoperability Vulnerabilities](#) by [Jake Miller](#) takes an in-depth look at how to trigger JSON parser inconsistencies, and where these usually-harmless quirks can become exploitable. Bundled Docker-based labs make these easy to replicate and practise.

6 - Cache Poisoning at Scale

Case studies make or break research, and [Cache Poisoning at Scale](#) has them in droves. [Youstin](#) proves [web cache poisoning](#) is still endemic, and still widely overlooked. DoS vulnerabilities are often spurned by researchers, but the persistent, single-request takedowns offered by web cache poisoning are clearly taken seriously by many companies. This is also a solid demonstration of the art of chaining tiny inconsistencies with secret headers and misconfigurations to concoct a severe vulnerability.

5 - Hidden OAuth attack vectors

Hackers typically focus on endpoints that are either directly visible, or discovered during recon. In [Hidden OAuth attack vectors](#), our own [Michael Stepankin](#) takes an alternative approach and dives deep into the [OAuth](#) and OpenID specifications to uncover hidden endpoints and design flaws that lay the stage for

enumeration, session poisoning and [SSRF](#). Michael has also updated both ActiveScan++ and Burp's discovery wordlists to keep an automated eye out and make sure this attack surface doesn't slip by unnoticed.

4 - Exploiting Client-Side Prototype Pollution in the wild

Described by filedescriptor as "arguably an underdog bug class as it's only occasionally exploited", Prototype Pollution was strictly a technique for enthusiasts until [A tale of making internet pollution free - Exploiting Client-Side Prototype Pollution in the wild](#) landed.

This phenomenal research defines a clear, insightful methodology for practical identification and exploitation. It's also notable for the all-star cast, headed by [s1r1us](#) - in Soroush's words *"It feels like watching Avengers!"*

3 - A New Attack Surface on MS Exchange

[Orange Tsai](#) is back in the Top 10 for the 5th year running, with the 3-part series [A New Attack Surface on MS Exchange](#). While most research focuses on uncovering classes of vulnerability common across numerous websites, this work instead delves astoundingly deep into a single target, with catastrophic results.

The entire panel loved this entry, describing it as a *"flawless intro to Exchange's architecture and attack surface, with reliable exploits and huge impact"*, an *"inspiring read if you want to start serious research"*, and a *"can of worms"* that *"changed the way many looked at this popular mailing solution and reminded us even the most secure looking apps can be broken easily if you are persistent and pay attention to all the details"*.

2 - HTTP/2: The Sequel is Always Worse

Nine months in the making, my own [HTTP/2: The Sequel is Always Worse](#) had a topic-collision with Emil's work above which made this more 'interesting' than it should have been, but some last-minute breakthroughs saved the day. Here's what the other panellists had to say: *"Ever wondered what could go wrong when converting between binary and ASCII protocols?" "This research has everything a reader needs. Besides the actual research and result, the quality write up, tooling, and the presentation make this very special." "This is a nice research on how HTTP2 tremendously increases the complexity of the whole situation. As HTTP2 usage is still being adopted, request smuggling will be even more relevant with the help of the never-ending HTTP (down)upgrade."*

If you enjoy this presentation, I highly recommend checking out the other high quality research papers on HTTP Request Smuggling in the full nomination list - just CTRL+F smuggling!

1 - Dependency Confusion

Some of the best research has an elegant simplicity that makes it seem deceptively obvious in hindsight. In [Dependency Confusion](#), [Alex Birsan](#) exposes critical design and configuration flaws affecting major package managers, exploiting package name ambiguity to achieve RCE on numerous major companies and earn well over \$100k in bounties. Beyond the crazy impact, it's also exceptionally well-explained, taking the reader through the entire research journey.

Discussions and mitigations are still underway for this attack, and we're really curious to see where this avenue of research goes next. Is the attack so elegant it can't be improved? Or is this just the humble beginning of a persistent new attack class? What we do know is if you're only going to read one piece of research this year, you should make it Dependency Confusion. Congratulations to Alex on a well deserved win!

Conclusion

2021 was a seriously good year for web security research.

More than ever this year, the top 10 list just scratches the surface and we recommend web security enthusiasts read the entire [nomination list](#). [Past year's top 10s](#) are well worth a look too! You can also get the drop on this year's top research the moment it's released by following [r/websecurityresearch](#) and [@PortSwiggerRes](#). Also, if you're interested in doing this kind of research yourself, I've [written up some advice for you](#). Also, before we wrap up, I should give an honorary mention to the repeatedly nominated, hot new hacking technique known only as 'F12'. Sadly it didn't make the cut.

Thanks again to everyone who took part! Without your nominations, votes, and most-importantly research, this wouldn't be possible.

Till next time!

top-10-techniques

Top 10 Hacking Techniques

← [Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering

 PortSwigger

 [Follow us](#)

© 2026 PortSwigger Ltd.

